

Trabajo Práctico 1 — Tu primera escena en Godot

Diplomatura de Videojuegos · Clase 1 Objetivo: armar, **vos solo/a**, una pequeña escena que junte casi todo lo que vimos en clase —nodos, escenas, familias 2D / UI, física y audio— sin escribir (casi) nada de código.

Qué vas a lograr

Al terminar vas a tener una escena que, al apretar :

- muestra un **piso** de color,
- deja **caer una caja** que rebota/aterriza sobre ese piso (física real),
- tiene un **personaje animado** parado en el nivel,
- una **cámara** encuadrando la acción,
- **música de fondo** que arranca sola,
- y un cartel de **UI** que dice “**Nivel 1**” pegado a la pantalla.

No hace falta que el personaje camine todavía: eso llega cuando veamos **GDScript**. Hoy el objetivo es **perderle el miedo al editor** y entender cómo se **arma y conecta** una escena.

 **Tiempo estimado:** 45–70 min. Leé cada paso completo antes de hacerlo.

Cómo va a quedar el árbol de nodos

Este es el **destino**. Tenelo a mano y andá comparando a medida que avanzás:

```

Nivel1 (Node2D)          ← raíz de la escena
├─ Piso (RigidBody2D)    ← 1
│  └─ Sprite2D
│  └─ CollisionShape2D
├─ Jugador (CharacterBody2D) ← 2
│  └─ AnimatedSprite2D
│  └─ CollisionShape2D
│  └─ Camera2D          ← 4 (cuelga del jugador)
├─ Caja (RigidBody2D)    ← 3
│  └─ Sprite2D
│  └─ CollisionShape2D
├─ Musica (AudioStreamPlayer) ← 5
└─ UI (CanvasLayer)     ← 6
   └─ Label ("Nivel 1")

```

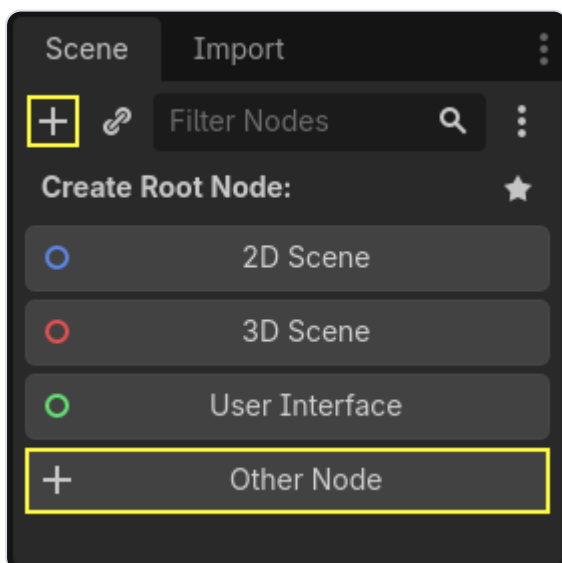
Cada número corresponde a una parte de este TP. **No saltees el orden**: cada parte usa lo de la anterior.

🔧 Parte 0 — Preparar el proyecto

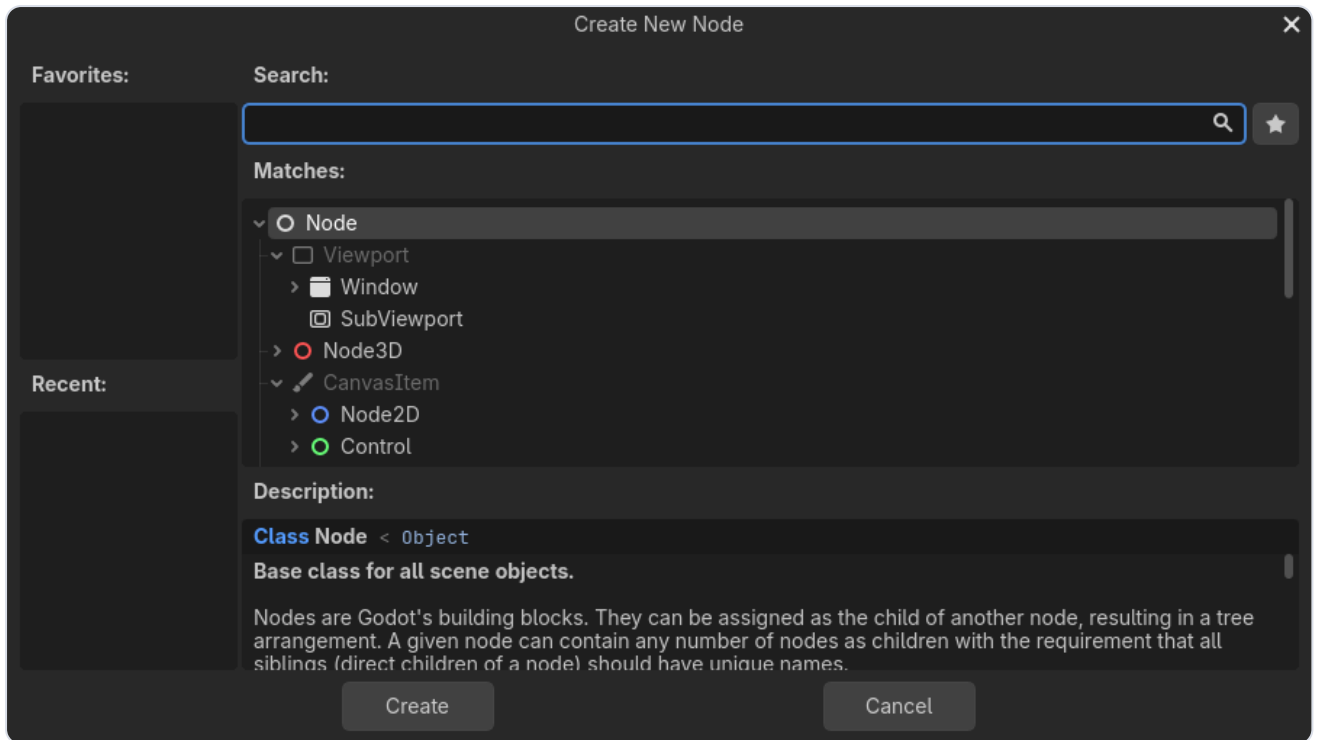
1. Abrió Godot. En el **Project Manager**, clic en **New Project** (Nuevo proyecto).
2. Nombre: `tp1-nivel`. Elegí una carpeta **vacía**. Dejé el renderizador en el que viene por defecto → **Create & Edit**.
3. Cuando abre el editor, arriba a la izquierda ves el panel **Escena (Scene)**. Todavía no hay ningún nodo.

Crear la escena y su raíz

4. En el panel **Escena**, entre las opciones rápidas, clic en **Otro Nodo** (*Other Node*).



5. Se abre el buscador de nodos. Escribí `Node2D`, seleccionalo y **Crear**.



6. En el panel Escena, **doble clic** sobre el nombre del nodo y renombralo a `Nivel1`.

Por qué `Node2D` como raíz: es el nodo base de todo lo 2D; aporta posición/rotación/escala en el plano. Todo lo demás va a **colgar** de él, formando el *árbol de escena* que vimos en clase.

7. Guardá con **Ctrl + S**. Nombre del archivo: `nivel1.tscn`. Fijate que aparece en el panel **FileSystem** (abajo a la izquierda). Recordá: las rutas del proyecto empiezan con `res://`.

✓ **Punto de control 0:** tenés un único nodo `Nivel1 (Node2D)` en el árbol y un archivo `nivel1.tscn` guardado.

■ Parte 1 — El Piso (RigidBody2D)

Conceptos: importar un sprite · modular su color · gravedad y cómo desactivarla.

1.1 · Crear el nodo del piso

1. Seleccioná `Nivel1` en el árbol.
2. Clic en el botón **+ Agregar Nodo Hijo** (*Add Child Node*) — el `+` arriba a la izquierda del panel Escena — o el atajo **Ctrl + A**. Se abre el mismo buscador de nodos de antes.
3. Buscá `RigidBody2D`, crealo, y renombralo a `Piso`.

🗨️ **¿Qué es un RigidBody2D?** Un cuerpo con **física real**: le afecta la gravedad, choca, rebota. Lo usamos acá **a propósito** para *ver* la gravedad en acción y después apagarla. (Ojo: un piso “de verdad” normalmente sería un `StaticBody2D` ; lo vemos como dato al final de esta parte.)

1.2 · Darle imagen: el Sprite2D

4. Con `Piso` seleccionado, agregá un hijo (**Ctrl + A**) → `Sprite2D` .
5. Necesitamos una imagen. Usá la que **ya viene con el proyecto**: en el panel **FileSystem** vas a ver `icon.svg` .
6. **Arrastrá** `icon.svg` desde FileSystem hasta la propiedad **Texture** del Sprite2D en el **Inspector** (derecha). Aparece el logo de Godot en el centro.

🗨️ **Importar = poner el archivo en la carpeta del proyecto.** Godot lo detecta e importa solo. Si más adelante querés otra imagen, copiá el archivo dentro de la carpeta del proyecto (o arrastralo al FileSystem) y listo.

1.3 · Convertirlo en plataforma y cambiarle el color

7. Con el **Sprite2D** seleccionado, en el Inspector buscá **Transform** → **Scale** y poné `x = 6` , `y = 0.6` . El logo se estira: ahora parece una plataforma ancha y finita.
8. Cambiá el color: en el Inspector, abrí **Visibility** → **Modulate** (dentro de la sección **CanvasItem**). Clic en el rectángulo de color y elegí un color, por ejemplo un verde.

🗨️ `Modulate` **es un tinte**. Multiplica el color del sprite por el que elijas: no “pinta” la imagen, la **tiñe**. `Self Modulate` hace lo mismo pero sin afectar a los nodos hijos. Es la forma más rápida de dar variedad sin abrir un editor de imágenes.

1.4 · Darle cuerpo físico: el CollisionShape2D

9. Seleccioná `Piso` de nuevo → agregá hijo (**Ctrl + A**) → `CollisionShape2D` .

⚠️ Si aparece un **triángulo amarillo** ⚠️ al lado del nodo, es normal: te avisa que **todavía no tiene forma**. Lo arreglamos ahora.

10. Con el **CollisionShape2D** seleccionado, en el Inspector buscá la propiedad **Shape** y hacé clic en `<vacío>` → **Nuevo RectangleShape2D**.
11. En el Viewport aparecen unos **puntos naranjas**: arrastralos hasta que el rectángulo **cubra la plataforma verde**. No tiene que ser perfecto, pero que tape bien la parte de arriba.

🗨️ **El Sprite es lo que se ve; el CollisionShape es lo que se toca.** Son dos cosas separadas a propósito: podés tener una imagen enorme con una colisión chiquita, o al revés. Si no ponés forma, **nada choca**.

1.5 · Ver la gravedad... y apagarla

12. Posicioná el piso: seleccioná `Piso` y, en el Inspector (**Transform** → **Position**), poné algo como `x = 0`, `y = 300` (recordá: en 2D, **la Y crece hacia abajo**).

13. Apretá  (**Play Scene**) arriba a la derecha, o **F6**.



Si te pide **guardar** o elegir escena principal, guardá `nivel1.tscn`.

14. **¿Qué pasa?** El piso **se cae de la pantalla** 😱. Es correcto: un `RigidBody2D` tiene gravedad. Cerrá la ventana del juego.

15. Ahora **apagá la gravedad**: seleccioná `Piso`, en el Inspector buscá la sección **RigidBody2D** y poné `Gravity Scale = 0`.

16. Además, para que quede **clavado como piso sólido** (que la caja no lo empuje), activá `Freeze` (Congelar). Si aparece **Freeze Mode**, dejalo en **Static**.

17. Volvé a apretar . Ahora el piso **se queda quieto**. 🎉

🗨️ Dos propiedades distintas:

- `Gravity Scale` = cuánta gravedad le afecta. En `0`, no cae.
- `Freeze` = lo "congela" en su lugar, inmóvil, aunque algo lo golpee.

💡 **Dato:** para un piso real que nunca se mueve conviene un `StaticBody2D` (no necesita nada de esto). Usamos `RigidBody2D` sólo para *entender* la gravedad. En la caja (Parte 3) la vamos a dejar **encendida** para verla caer.

✅ **Punto de control 1:** al dar Play, ves una plataforma verde **quieta** en la parte de abajo.

■ Parte 2 — El Jugador (`CharacterBody2D` + animación)

Conceptos: `CharacterBody2D` · `AnimatedSprite2D` con `SpriteFrames` · `CollisionShape2D`.

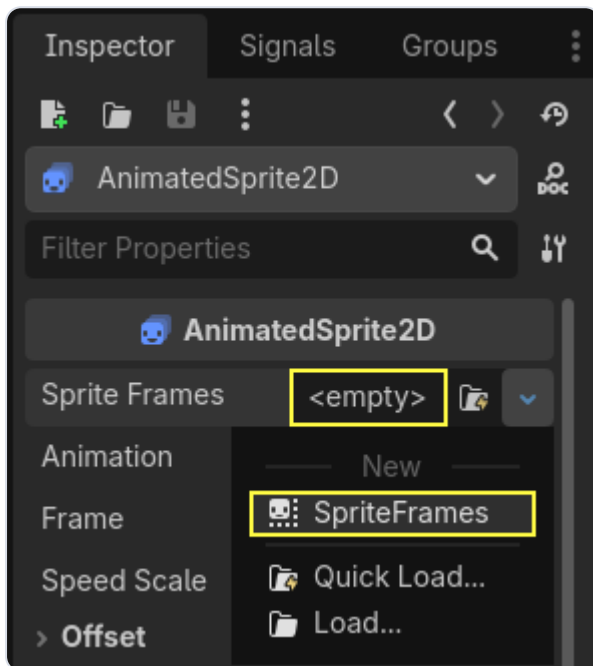
2.1 · El nodo del jugador

1. Seleccioná `Nivel1` → agregá hijo (**Ctrl + A**) → `CharacterBody2D` → renombralo `Jugador`.

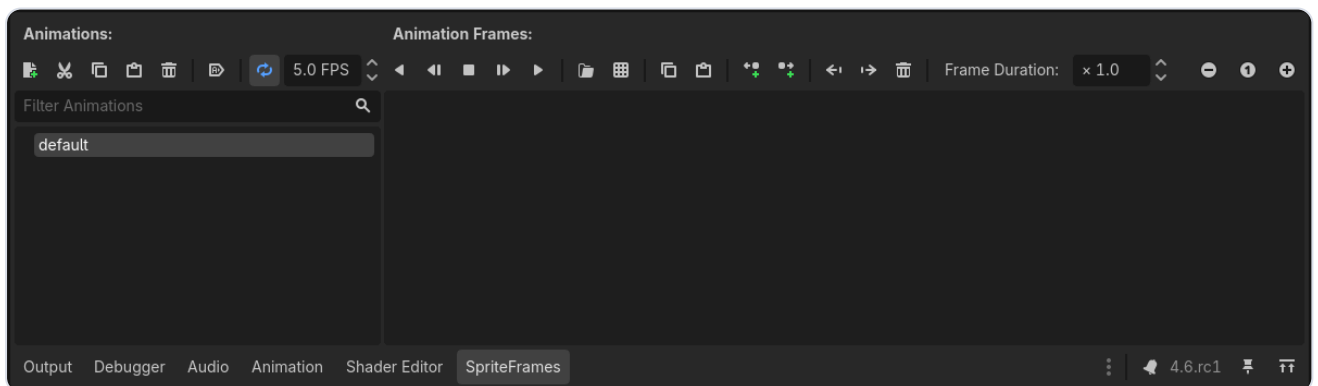
CharacterBody2D es el nodo pensado para **personajes controlados**. A diferencia del **RigidBody2D**, **no se mueve solo**: se mueve por código. Por eso hoy va a quedar **flotando quieto** donde lo pongas — y ese es el gancho para la clase de GDScript. 😊

2.2 · La animación: AnimatedSprite2D + SpriteFrames

- Con **Jugador** seleccionado, agregá hijo → **AnimatedSprite2D**.
- En el Inspector, buscá la propiedad **Sprite Frames**, clic en **<vacío>** → **Nuevo SpriteFrames**.



- Clic sobre el recurso **SpriteFrames** que acabás de crear: abajo se abre el panel **SpriteFrames**.



- Ahí vas a cargar los **cuadros** de la animación. Tenés dos caminos:

Camino A — con un sprite sheet (recomendado):

- Descargá un personaje gratis (CC0) de kenney.nl/assets (por ejemplo "Pixel Platformer" o "Platformer Characters").
- Copió las imágenes dentro de la carpeta del proyecto (arrastralas al **FileSystem**).
- En el panel **SpriteFrames**, usá **+** (**Añadir cuadros desde un archivo**) y elegí las imágenes de la animación de caminar/idle. Se van agregando como cuadros.

Camino B — sin descargar nada (plan B):

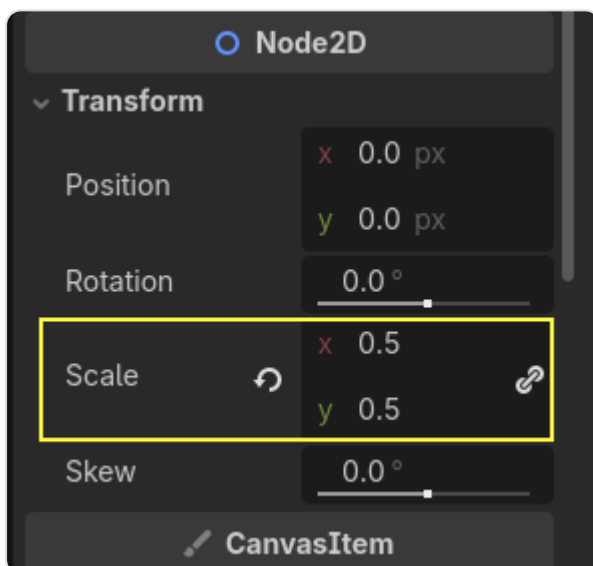
- En el panel SpriteFrames, agregá **el mismo** `icon.svg` **dos veces** como dos cuadros.
- En el Inspector del Sprite (o con `Modulate`) hacé que se vea distinto, o simplemente dejá los dos cuadros iguales: la idea es **ver el sistema funcionar**, no que sea lindo.

6. Renombrá la animación (arriba en el panel) a `idle` si querés, y **subí los FPS** a **~6** para que se note.

7. **Que arranque sola y en loop:** en el panel SpriteFrames activá el botón  **Loop** y el botón **Autoplay on Load** (el ícono con una **A**). Así la animación se reproduce **sin código**.

 `SpriteFrames` es un recurso que guarda **listas de cuadros** (una por animación: idle, correr, saltar...). El `AnimatedSprite2D` es el nodo que las **muestra**. Es el equivalente animado del `Sprite2D`.

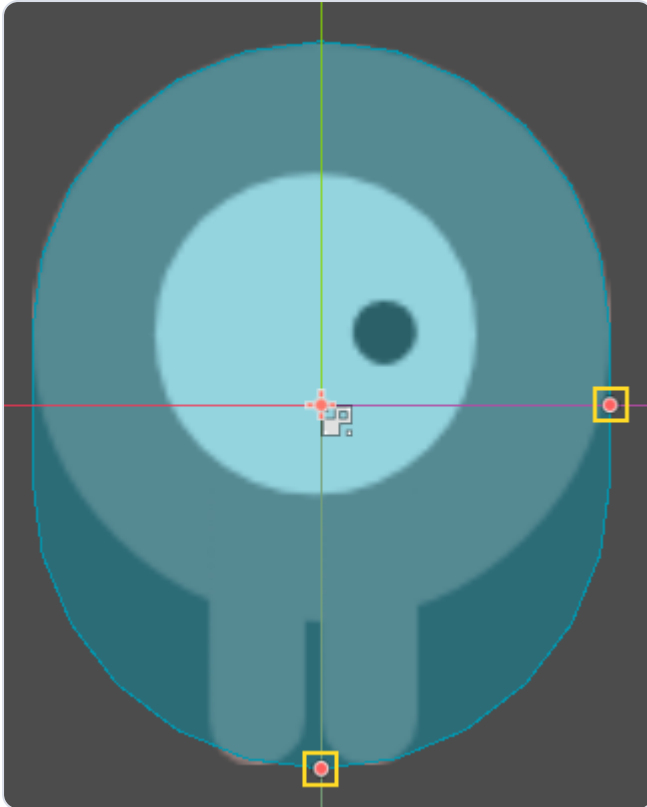
8. Si el personaje se ve gigante, seleccioná el **AnimatedSprite2D** y bajá **Transform** → **Scale** a algo como `0.5, 0.5`.



2.3 · Su colisión

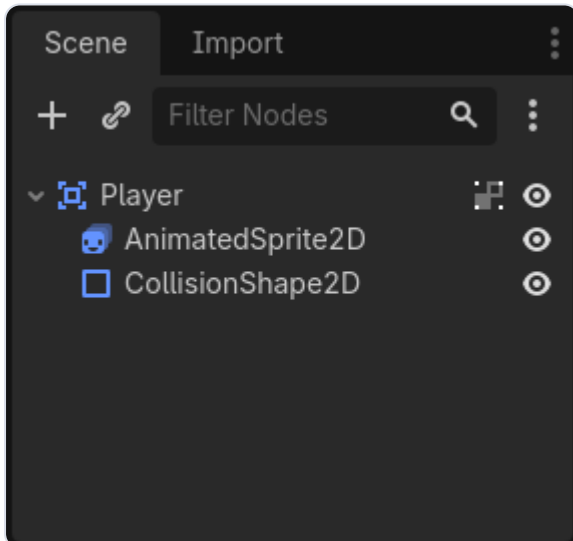
9. Seleccioná `Jugador` → agregá hijo → `CollisionShape2D`.

10. En el Inspector → **Shape** → `<vacío>` → **Nuevo CapsuleShape2D** (una cápsula va bien para un cuerpo). Ajustá los puntos naranjas para que **envuelva** al personaje.



11. Posicioná al **Jugador** arriba del piso (por ejemplo **Position** `x = 0` , `y = 0`).

✓ **Punto de control 2:** al dar  , ves al personaje **animándose** (aunque quieto en el lugar). Debería quedar parecido a esto:



Parte 3 — La Caja que cae (RigidBody2D)

Conceptos: `RigidBody2D` con **gravedad encendida** · `Sprite2D` · `CollisionShape2D` chocando contra el piso.

1. Seleccioná **Nivel1** → agregá hijo → `RigidBody2D` → renombralo **Caja** .

- Hijo de **Caja** → **Sprite2D** → arrastrale **icon.svg** a **Texture**. Bajale la escala a algo chico, tipo **0.4**, **0.4**. (Opcional: **Modulate** marrón para que parezca caja.)
- Hijo de **Caja** → **CollisionShape2D** → **Shape** → **Nuevo RectangleShape2D** → ajustá el rectángulo al tamaño del sprite.
- Posicioná la **Caja** **bien arriba y sobre el piso**: por ejemplo **Position** **x = 0**, **y = -200**.
- Importante**: esta vez **NO** toques **Gravity Scale** ni **Freeze**. Queremos que **caiga**.
- Apretá . La caja **cae por gravedad y aterriza sobre la plataforma verde**. 🎉

💡 Esto es lo que hace un motor por vos: **no programaste la caída ni el choque**. El **RigidBody2D** + los **CollisionShape2D** resuelven la física solos. Sin motor, esto serían muchas horas de matemática.

🔗 ¿La caja atraviesa el piso o no aparece?

▼ Abrí para ver soluciones

- ¿El **piso** tiene su **CollisionShape2D** con **forma** asignada? Sin forma, no choca.
- ¿La **caja** tiene su **CollisionShape2D** con forma?
- ¿La caja arranca **arriba** del piso (Y más chica) y **alineada** en X?
- ¿Dejaste el piso con **Freeze = On**? Si no, la caja lo empuja hacia abajo.

✅ **Punto de control 3**: la caja cae y **se queda apoyada** sobre el piso.

📺 Parte 4 — La Cámara (Camera2D)

Concepto: **Camera2D** siguiendo al jugador — **sin script**.

- Seleccioná **Jugador** → agregá hijo (**Ctrl + A**) → **Camera2D**.
- En el Inspector, asegurate de que **Enabled** (Habilitado) esté tildado. Si es la única cámara, se vuelve **la activa** automáticamente.
- (Opcional) Si ves todo muy chico o muy grande, ajustá **Zoom** (por ejemplo **2**, **2** para acercar).
- Apretá : ahora la vista está **centrada donde está el jugador**.

💡 ¿Por qué “sigue” al jugador si no hay código? Porque la **colgamos como hija del Jugador**. En clase vimos que **el hijo hereda la transformación del padre**: si el jugador se moviera, la cámara se movería con él. Como todavía no se mueve, queda fija — pero la estructura ya es la correcta.

✅ **Punto de control 4**: la cámara encuadra la escena centrada en el jugador.

🎵 Parte 5 — La Música (AudioStreamPlayer)

Concepto: `AudioStreamPlayer` con música de fondo en **autoplay**.

1. Conseguí un archivo de música corto (`.ogg` o `.mp3`). Fuentes gratis (CC0): kenney.nl/assets (sección Audio) o incompetech.com. Si ya tenés un `.mp3` a mano, sirve.
2. **Copió el archivo dentro de la carpeta del proyecto** (arrastralo al **FileSystem**).
3. Seleccioná `Nivel1` → agregó hijo → `AudioStreamPlayer` → renombralo `Musica` .
4. En el Inspector, propiedad **Stream**: arrastrá tu archivo de música desde el FileSystem hasta ahí.
5. Tildá la propiedad `Autoplay` (Reproducción automática).
6. Apretá : la música **arranca sola**. 

 `AudioStreamPlayer` reproduce sonido "global" (sin posición). Para efectos ubicados en el mundo (pasos, disparos) existe `AudioStreamPlayer2D` . `Autoplay` = "empezá apenas aparezca la escena".

🔧 ¿No suena?

▼ Abrió para ver soluciones

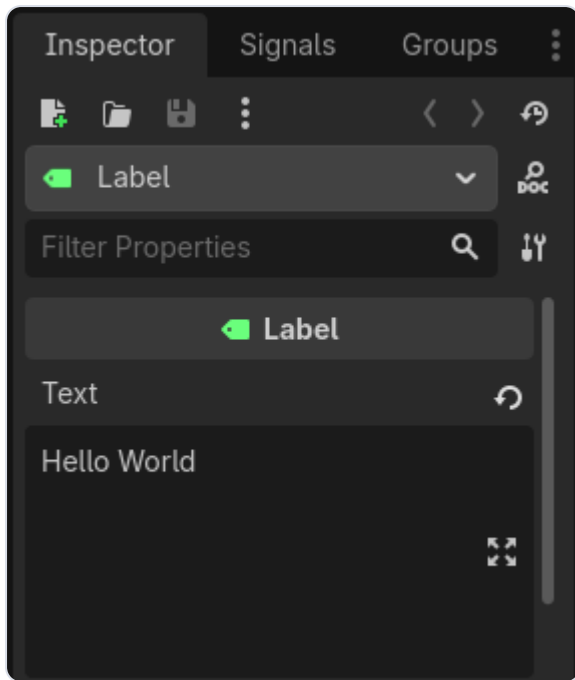
- ¿El archivo está **dentro** de la carpeta del proyecto (lo ves en el FileSystem)?
- ¿Lo arrastraste a la propiedad **Stream** y quedó cargado?
- ¿Está tildado **Autoplay**?
- Subí el volumen de la compu 😊 . Para que **loopee**: seleccioná el archivo `.ogg` en el FileSystem → pestaña **Import** (arriba) → tildá **Loop** → **Reimportar**.

✅ **Punto de control 5:** se escucha música al dar Play.

🟩 Parte 6 — La UI: cartel "Nivel 1" (CanvasLayer + Label)

Conceptos: `CanvasLayer` para UI fija · `Label` con texto.

1. Seleccioná `Nivel1` → agregó hijo → `CanvasLayer` → renombralo `UI` .
2. Con `UI` seleccionado, agregó hijo → `Label` .
3. Con el **Label** seleccionado, en el Inspector buscá la propiedad **Text** y escribí `Nivel 1` .



4. Moví el Label a una esquina (por ejemplo arriba a la izquierda) con la herramienta de mover, o desde **Transform** → **Position**.
5. (Opcional) Agrandé la letra: Inspector → **Theme Overrides** → **Font Sizes** → **Font Size**.
6. Apreté . El cartel “**Nivel 1**” queda **fijo en la pantalla**, aunque la cámara se mueva.

💡 **¿Por qué va dentro de un `CanvasLayer`?** Porque la UI **no vive en el mundo, vive en la pantalla** (lo vimos en clase). El `CanvasLayer` es una capa aparte que **no se mueve con la cámara**. Ese cartel “Nivel 1” es un ejemplo de UI **no diagética**: existe para vos, el personaje no lo ve.

✅ **Punto de control 6:** ves “Nivel 1” fijo en pantalla mientras corre todo lo demás.

🏁 Resultado final

Apreté  una última vez. Deberías tener, todo junto:

-  piso quieto,
-  caja que cae y aterriza,
-  personaje animado,
-  cámara encuadrando,
-  música de fondo,
-  cartel “Nivel 1”.

Guardá con Ctrl + S. ¡Terminaste tu primera escena! 🎉

Entrega

Entregá **una** de estas opciones (según indique el/la docente):

1. La **carpeta del proyecto** comprimida en `.zip` (sin la carpeta `.godot/`), o
2. Una **captura de pantalla** del juego corriendo + una captura del **árbol de nodos** (panel Escena).

Nombre del archivo: `tp1-nivel-ApellidoNombre.zip`

✓ Checklist de autoevaluación

Antes de entregar, confirmá que:

- ☐ La raíz es un `Node2D` llamado `Nivel1` y guardaste `nivel1.tscn`.
- ☐ El **Piso** es `RigidBody2D`, tiene `Sprite2D` (con color por `Modulate`) y `CollisionShape2D`, y **no se cae** (`Gravity Scale = 0` + `Freeze`).
- ☐ El **Jugador** es `CharacterBody2D` con `AnimatedSprite2D` (animación que **se reproduce sola**) y `CollisionShape2D`.
- ☐ La **Caja** es `RigidBody2D`, **cae** y **aterriza** sobre el piso.
- ☐ La `Camera2D` **cuelga del Jugador**.
- ☐ Hay `AudioStreamPlayer` con música en **Autoplay**.
- ☐ Hay un `CanvasLayer` con un `Label` que dice **"Nivel 1"**.

Extra (opcional, para los que quieran más)

- **Que el personaje caiga y camine:** eso necesita un pequeño script en el `Jugador` — lo vemos en la próxima clase (GDScript). Si te animás a espiar: mirá `_physics_process()` y `move_and_slide()` en la doc.
- **Más cajas:** duplicá la `Caja` (**Ctrl + D**) y movelas: vas a ver la física apilándolas.
- **Piso inclinado:** rotá el `Piso` unos grados (**Transform** → **Rotation**) y mirá cómo la caja **resbala**.

Recursos

- Documentación oficial (pasos y capturas de este TP): docs.godotengine.org/es/4.x — **Primeros pasos**
- Assets gratis (CC0): kenney.nl/assets
- Música libre: incompetech.com

Las capturas de este documento provienen de la **documentación oficial de Godot Engine** (Juan Linietsky, Ariel Manzur y la comunidad), bajo licencia **CC BY 4.0**.